

# React + Firebase Authentication

Protected Routes — Complete Study & Reference Documentation

## 1. Objective

The goal is to make selected React routes accessible only to authenticated users. In this example, `/login` and `/signup` are public routes, while `/` (Home) is protected.

### Expected behavior:

User visits `/` → authentication is checked → authenticated user sees Home; unauthenticated user is redirected to `/login`.

## 2. Authentication vs Authorization

**Authentication:** “Who is the user?” Firebase Authentication handles sign-in, sign-up, and the authenticated user session.

**Authorization:** “Is this user allowed to access this page/resource?” A `ProtectedRoute` can enforce this routing rule on the client.

## 3. Basic Application Flow

User → React Router → `ProtectedRoute` → Firebase Auth → (User exists → Home) OR (No user → `/login`)

## 4. Firebase’s Role

Firebase Authentication maintains the authentication state. The application listens for that state using `onAuthStateChanged()`. Firebase Web Auth also supports persistence, so the authenticated state can survive a refresh/browser restart depending on the configured persistence.

## 5. React State’s Role

React state represents the current authentication result inside the React application:

```
const [user, setUser] = useState(null)
```

Important: React state itself does **not** permanently save the login. On refresh, React state is recreated. Firebase Auth persistence is what allows Firebase to restore the authenticated user, after which `onAuthStateChanged()` updates React state.

## 6. Why a Loading State Is Necessary

Use **loading** to distinguish “Firebase has not finished checking” from “there is no logged-in user.”

```
const [loading, setLoading] = useState(true)
```

Initial state: `loading = true`, `user = null`. After Firebase responds, `setUser(user)` stores the result and `setLoading(false)` tells the app that the check is complete. Without this waiting period, an application can incorrectly redirect a still-authenticated user during initialization.

## 7. Understanding useEffect + onAuthStateChanged

This is the core code used to connect Firebase Auth with React:

```
useEffect(() => {  
  
  const unsubscribe = onAuthStateChanged(auth, (user) => {  
  
    setUser(user)  
    setLoading(false)  
  
  })  
  
  return () => unsubscribe()  
  
}, [auth])
```

## 8. Line-by-Line Explanation

### **useEffect(() => { ... }, [auth])**

Runs the Firebase listener setup when the ProtectedRoute component is mounted and manages it according to the dependency array.

### **onAuthStateChanged(auth, callback)**

Registers an authentication-state listener. Firebase calls the callback with a User object when authenticated, or null when no user is authenticated.

### **(user) => { ... }**

The user value is supplied by Firebase. It is not a manually created variable containing the login result.

### **setUser(user)**

Copies Firebase's current authentication result into React state so the component can make a routing decision.

### **setLoading(false)**

Marks the initial Firebase authentication check as complete.

### **const unsubscribe = ...**

Stores the unsubscribe/cleanup function returned by the Firebase listener.

### **return () => unsubscribe()**

Removes the Firebase listener when the effect is cleaned up, preventing an unnecessary active subscription.

### **[auth]**

The dependency array tells React what values the effect depends on.

## 9. ProtectedRoute Component

```
import React, { useEffect, useState } from 'react'  
import { Navigate } from 'react-router'  
import { getAuth, onAuthStateChanged } from 'firebase/auth'  
import app from '../firebase/config'  
  
function ProtectedRoute({ children }) {
```

```

const [user, setUser] = useState(null)
const [loading, setLoading] = useState(true)

const auth = getAuth(app)

useEffect(() => {

  const unsubscribe = onAuthStateChanged(auth, (user) => {
    setUser(user)
    setLoading(false)
  })

  return () => unsubscribe()

}, [auth])

if (loading) {
  return Loading...
}

if (!user) {
  return
}

return children
}

export default ProtectedRoute

```

## 10. Understanding children

When you write `<ProtectedRoute><Home /></ProtectedRoute>`, the Home component is the **children** prop. Therefore, **return children** means “the user passed the authentication check, so render the protected page.”

## 11. Using ProtectedRoute with React Router

```

path="/"
element={

}
/>

} />

} />

```

## 12. Route Behavior

| URL     | Authentication | Result                                  |
|---------|----------------|-----------------------------------------|
| /login  | Not required   | Login page                              |
| /signup | Not required   | Signup page                             |
| /       | Required       | Home if authenticated; otherwise /login |

## 13. Directly Visiting /

If a user manually enters `http://localhost:5173/` in the browser, React Router matches the `/` route, but `ProtectedRoute` gets the final say about rendering the `Home` component.

**Unauthenticated:** `/` → `ProtectedRoute` → `user = null` → `<Navigate to="/login" />` → `/login`

**Authenticated:** `/` → `ProtectedRoute` → Firebase user exists → return children → `<Home />`

## 14. What Happens on Refresh?

This is one of the most important concepts. After refresh, React state is recreated, so the initial `user` state can be null. Firebase Auth persistence can restore the previous authentication state. Then `onAuthStateChanged()` provides the restored user to React, `setUser(user)` updates the state, and the `ProtectedRoute` renders `Home`.

Refresh → React initializes → `loading = true` → Firebase restores/checks Auth → `onAuthStateChanged()` → `setUser(user)` → `setLoading(false)` → `ProtectedRoute` decides → `Home`

## 15. Firebase Auth Persistence

**LOCAL:** Authentication persists across browser sessions until sign-out (default for many Firebase Web Auth setups).

**SESSION:** Authentication persists for the current session/tab and is cleared when the session ends.

**NONE:** Authentication is kept only in memory and is not persisted.

These persistence choices control whether Firebase can restore the authentication state after a refresh/session restart. They are separate from React `useState`.

## 16. Authentication vs React State — Final Mental Model

Firebase Auth → maintains authentication/persistence → `onAuthStateChanged()` → gives current user → React `setUser()` → `ProtectedRoute` checks user → `Home` OR `/login`

## 17. Quick Reference

| Code                                        | Meaning                                       |
|---------------------------------------------|-----------------------------------------------|
| <code>useState(null)</code>                 | React-side representation of the current user |
| <code>useState(true)</code>                 | Wait for the initial auth check               |
| <code>getAuth(app)</code>                   | Get Firebase Auth instance                    |
| <code>onAuthStateChanged()</code>           | Listen for authentication state               |
| <code>setUser(user)</code>                  | Store Firebase's user result in React state   |
| <code>setLoading(false)</code>              | Authentication check is finished              |
| <code>unsubscribe()</code>                  | Remove Firebase auth listener                 |
| <code>&lt;Navigate to="/login" /&gt;</code> | Redirect unauthenticated user                 |
| <code>return children</code>                | Render protected content                      |

## 18. Important Note for Future Projects

This ProtectedRoute pattern is a good learning foundation. As the application grows, a central AuthContext or authentication provider is usually cleaner than registering the same Firebase listener separately in many components. The underlying concepts remain the same: Firebase Auth is the source of authentication state, React consumes that state, and routing decides what UI to expose.

## 19. Official Documentation for Further Reference

Firebase Auth state changes: <https://firebase.google.com/docs/auth/web/manage-users>

Firebase Auth persistence: <https://firebase.google.com/docs/auth/web/auth-state-persistence>

React useEffect: <https://react.dev/reference/react/useEffect>

React Router Navigate: <https://reactrouter.com/api/components/Navigate>